

RGB Social Navigation BEV 任務報告

CPU-only 單目 RGB 社會導航感知、BEV 投影與風險可視化

SegFormer walkable segmentation

YOLO11 object detection

Kalman + Hungarian tracking

Homography / IPM BEV

Occupancy grid

實驗影片

`data/15659193_3840_2160_60fps.mp4`

影片規格

3840 x 2160, 約 59.94 FPS, 763 frames

執行方式

以指定影片抽樣推論，產生實際辨識結果圖與處理時間統計

輸出檔案

`reports/rgb_social_navigation_bev_report.pdf`

摘要

本任務建立並驗證一套以單目 RGB 影像為輸入的社會導航 BEV 感知系統。系統能從影片或即時相機逐幀讀取 RGB frame，進行可行走區域分割、人員與障礙物偵測、多人追蹤、未知障礙物估測，最後透過 homography / IPM 投影至鳥瞰圖，輸出 social zone、occupancy grid、2x2 視覺化影片與逐幀 JSONL 結果。

本報告使用 `data/15659193_3840_2160_60fps.mp4` 進行抽樣測試。由於目前未提供實際場地校正檔，BEV 結果以非公尺尺度顯示；若要量測實際距離與 social zone 半徑，需先建立 `configs/calibration.yaml`。

1. 任務目標與使用情境

此系統的目標不是取代深度相機或 LiDAR，而是在只有 RGB 相機的條件下，提供社會導航所需的環境理解與風險可視化資訊。主要使用情境包含：

- 從即時相機或影片取得 RGB 影像，線上逐幀推論，不需要先離線處理整支影片。
- 標示行人位置、追蹤 ID、行人移動軌跡與周圍 social safety zone。
- 把前視角感知結果投影成 BEV occupancy grid，供導航、人機互動或報告分析使用。
- 輸出 2x2 視覺化影片、JSONL 結果與 NPY/PNG occupancy grid。

安全定位：此系統是 perception / risk visualization 工具，不是安全認證控制器。單目 RGB 無法直接取得真實深度，未知障礙物也是低信心 RGB 估測，不應直接作為機器人安全停機邏輯。

2. 系統架構

系統由輸入層、runtime 控制、CPU perception pipeline、BEV 投影與輸出層構成。GUI 提供 Start / Pause / Stop 與 realtime playback；Pause 會在下一張 frame 進入推論前停住，因此不會繼續累積離線預算。

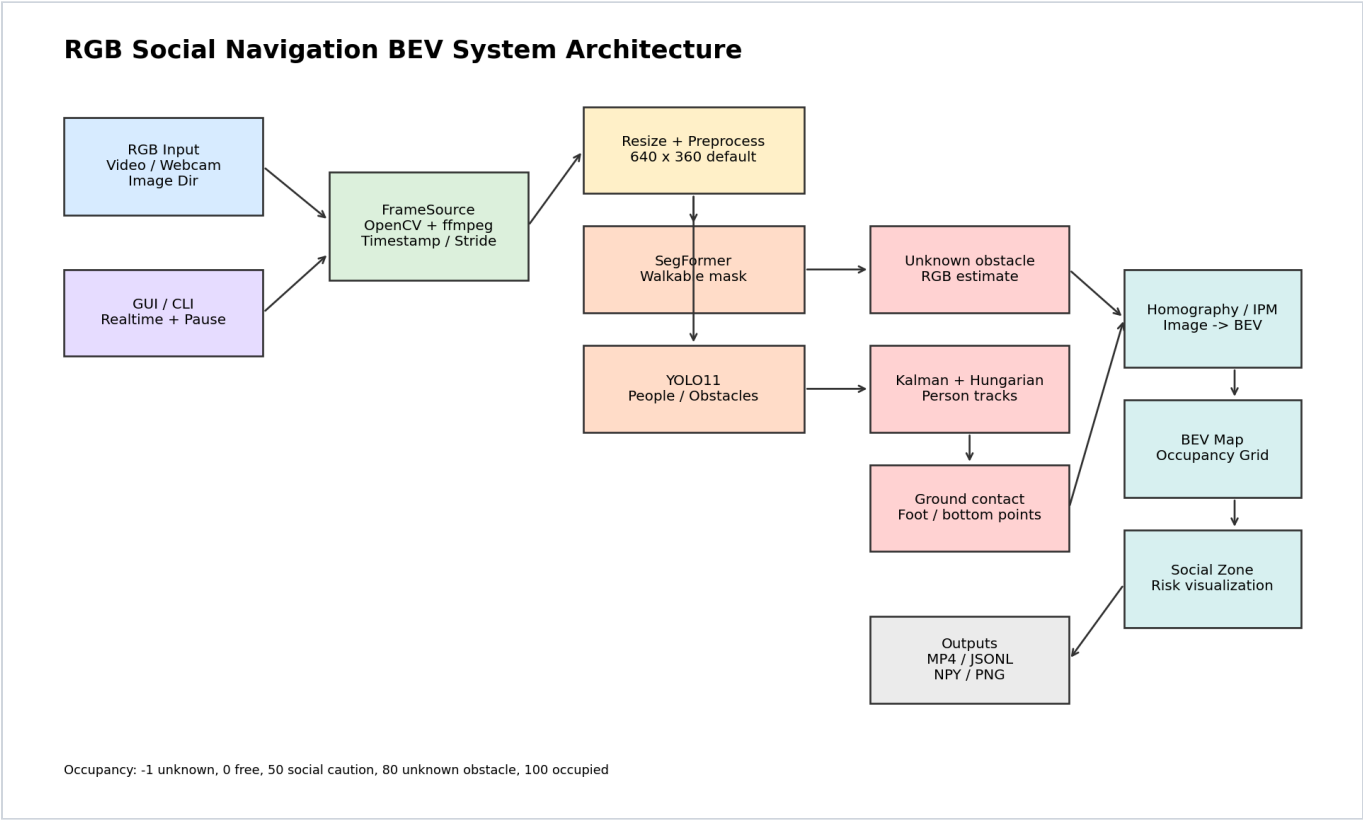


圖 1. RGB Social Navigation BEV 系統架構。

核心模組

模組	功能	實作檔案
FrameSource	支援 webcam、影片、圖片資料夾與單張圖片；OpenCV 讀取失敗時可用 ffmpeg fallback。	social_bev/frame_source.py
WalkableSegmenter	使用 SegFormer 產生可行走區域 mask，模型失敗時可退回 RGB heuristic。	social_bev/segmentation.py
ObjectDetector	使用 YOLO11 偵測 person 與已知障礙物；無 OpenVINO IR 時使用 yolov11n.pt CPU fallback。	social_bev/detection.py

MultiObjectTracker	以 Kalman filter 與 Hungarian matching 維持 person track ID 與短期軌跡。	<code>social_bev/tracking.py</code>
BEVMapBuilder	整合 walkable mask、obstacle、track 與 social zone，產生 BEV image 與 occupancy grid。	<code>social_bev/bev_map.py</code>

3. 實驗設定

輸入影片	data/15659193_3840_2160_60fps.mp4
影片解析度 / FPS	3840 x 2160 / 59.94 FPS
抽樣方式	每 60 frames 取 1 frame，共處理 8 個 sample。
Pipeline input size	640 x 360
Segmentation	torch backend, SegFormer ADE20K
Detection	openvino configured; runtime fallback to yolov11n.pt because OpenVINO YOLO IR is not present.
Seg / Det interval	segmentation interval = 2, detection interval = 2
Calibration	未提供 calibration YAML，因此使用 NON-METRIC BEV。

說明：抽樣測試用於產生代表性報告圖與效能數據。若完整處理 763 frames，CPU 環境會耗費較長時間，且不代表即時相機的低延遲設定。

4. 實際辨識結果

以下圖片由指定影片實際執行 pipeline 取得。2x2 視覺化依序包含：原圖偵測與 track ID、walkable mask 疊圖、障礙物/ground contact、BEV occupancy 與 social zone。



圖 2. Sample 1 的 2x2 pipeline visualization。



圖 3. Sample 8 的 2x2 pipeline visualization。

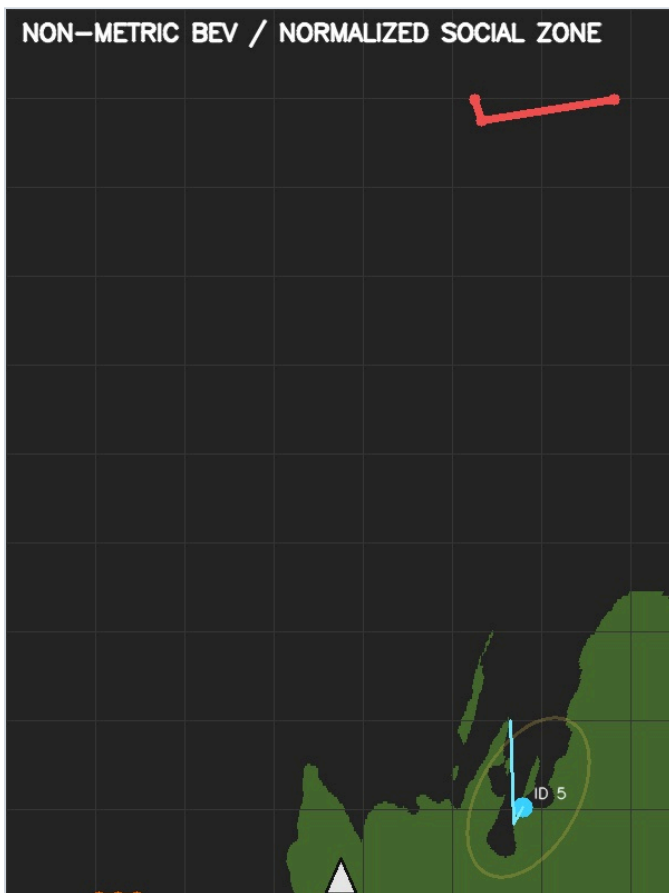


圖 4. Sample 8 的 BEV map 與 social zone。



圖 5. Sample 8 的 occupancy grid PNG preview。灰色代表 unknown。

5. 效能與數據分析

處理 sample 數	8
總 wall time	19.08 s
Wall average FPS	0.419 FPS，包含模型初始化與抽樣 decode。
Mean processing time	1153.5 ms/frame
Median processing time	346.7 ms/frame
Pipeline mean FPS	16.44 FPS，受 interval reuse 影響。
平均人員 track 數	5.00，最多 8
平均 unknown obstacle 數	1.25，最多 3
平均 walkable ratio	7.01%

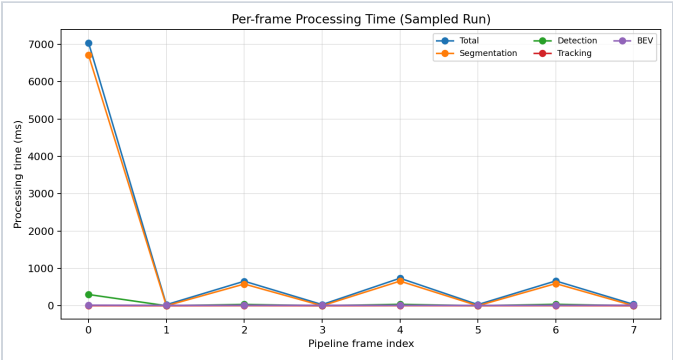


圖 6. 各模組逐幀處理時間。第 0 frame 包含模型載入與 warm-up，因此較高。

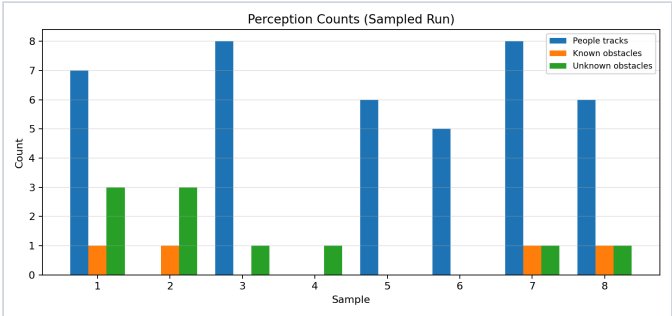


圖 7. 每個 sample 的 people / known obstacle / unknown obstacle 數量。

Frame	Time (s)	People tracks	Known obs.	Unknown obs.	Walkable	Total ms	FPS
0	0.00	7	1	3	3.3%	7042.5	0.14
1	1.00	0	1	3	3.3%	28.6	34.91
2	2.00	8	0	1	3.3%	656.8	1.52
3	3.00	0	0	1	3.3%	31.1	32.20
4	4.00	6	0	0	9.3%	737.0	1.36
5	5.00	5	0	0	9.3%	30.7	32.53
6	6.01	8	1	1	12.1%	664.6	1.50
7	7.01	6	1	1	12.1%	36.5	27.39

Realtime 判讀：系統架構是線上逐幀處理，可直接接 webcam；但 CPU SegFormer + YOLO 不保證達到 60 FPS。若用於即時相機，建議採用較小 input size、提高 segmentation/detection interval、匯出 OpenVINO 模型，並加入 latest-frame mode 以丟棄過期 frame，避免累積延遲。

6. Occupancy Grid 與輸出格式

數值	語意	用途
----	----	----

-1	unknown	RGB 無法確認或未投影區域。
0	free	由 walkable mask 投影得到的可行走區域。
50	social caution	行人周圍 social zone，導航應提高避讓權重。
80	unknown obstacle	低信心 RGB 未知障礙物估測。
100	occupied	人、已知障礙物或 robot footprint。

主要輸出包含 `outputs/*.mp4` 2x2 視覺化影片、`outputs/*.jsonl` 逐幀結構化資料，以及 `outputs/*_occupancy/frame_*.npy/.png` 的 occupancy grid。

7. 限制與改善方向

- **單目 RGB 無真實深度**：BEV 投影依賴 ground-plane homography；非平面地面、坡度、懸空物體與遮擋會造成位置誤差。
- **未校正時不具公尺尺度**：本次未提供 calibration YAML，因此 social zone 是 normalized pixel zone。若要估算實際距離，需使用 `tools/calibrate_ipm.py` 完成場地校正。
- **未知障礙物為低信心估測**：目前由非 walkable 區域與 corridor heuristic 推出，可能受陰影、行人遮擋與地面材質影響。
- **CPU 即時性有限**：本系統可接即時相機，但不保證等於相機原生 FPS。實務上應以低延遲為目標，必要時跳 frame 或只保留最新 frame。
- **模型加速**：建議匯出 YOLO 與 SegFormer OpenVINO IR，並調整 `input_width/input_height`、`segmentation_interval`、`detection_interval`。

8. 執行指令

GUI 一鍵啟動

```
cd /home/laiting/Desktop/NTHU_HRC_lab/itri_safty_descision/rgb_social_navigation_bev
python gui.py
```

指定影片 CLI

```
cd /home/laiting/Desktop/NTHU_HRC_lab/itri_safty_descision/rgb_social_navigation_bev
python -m social_bev.run --input data/15659193_3840_2160_60fps.mp4 --config configs/default.yaml --output outputs/real_social_navigation_result.mp4 --jsonl outputs/real_social_navigation_results.jsonl --occupancy-dir outputs/real_social_navigation_occupancy --device cpu
```

即時相機

```
python -m social_bev.run --input 0 --config configs/default.yaml --output outputs/webcam_social_bev.mp4 --jsonl outputs/webcam_social_bev.jsonl --occupancy-dir outputs/webcam_social_bev_occupancy --device cpu
```

9. 結論

本系統已完成從 RGB frame source 到 social navigation BEV visualization 的端到端流程，能在 CPU 環境下對真實人群影片進行 walkable segmentation、person detection/tracking、unknown obstacle estimation、BEV occupancy rendering 與 JSONL/影片輸出。實驗結果顯示系統可線上逐幀運作並產生可檢視的社會導航風險圖，但若部署於即時機器人，需要進一步完成 metric calibration、模型加速與 latest-frame 低延遲策略。